Deploy a Slack Marketplace App on GCP e2-micro with Public HTTPS Endpoints

This guide shows how to deploy a Slack app for the Slack Marketplace/App Directory on a Google Cloud Platform `e2-micro` VM in `us-central1`.

This is different from the existing Socket Mode setup because a public Slack app must use HTTP request URLs and a publicly reachable HTTPS endpoint.

Why this guide is different

Slack Marketplace submission requires:

- a public HTTPS endpoint
- verified Request URLs for event subscriptions and interactivity
- public distribution enabled in the Slack app settings

For this reason, the deployment should expose a web service that can receive Slack events over HTTP rather than relying only on Socket Mode.

---

1. Create the VM instance

1. Open the Google Cloud Console.
2. Go to Compute Engine → VM instances → Create instance.
3. Use these values:
    - Name: `pinionai-slack-marketplace`
    - Region: `us-central1`
    - Zone: `us-central1-a` or another `us-central1-*` zone
    - Machine type: `e2-micro`
    - Boot disk: Debian 12
4. Under Firewall, allow HTTP and HTTPS traffic.
5. Make sure you have a static or predictable external IP address.

Free-tier notes

Use the same free-tier-safe settings described in the existing GCP deployment guide:

- Debian 12
- Standard persistent disk
- 30 GB disk size
- `e2-micro`

---

2. Connect to the VM

Use SSH to connect:

```
gcloud compute ssh pinionai-slack-marketplace --zone=us-central1-a
```

If you prefer, you can also use the browser-based SSH option from the GCP console.

---

3. Prepare the VM

Update the system and add swap memory for the small machine size:

```
sudo apt update && sudo apt upgrade -y
sudo fallocate -l 2G /swapfile
sudo chmod 600 /swapfile
sudo mkswap /swapfile
sudo swapon /swapfile
echo '/swapfile none swap sw 0 0' | sudo tee -a /etc/fstab
```

Install required packages:

```
sudo apt install -y python3 python3-venv python3-pip git curl build-
essential nginx certbot python3-certbot-nginx
```

---

4. Clone the repository

```
cd /home/$USER
git clone https://github.com/pinionai/pinionai-streamlit-agent.git
cd pinionai-streamlit-agent
```

---

5. Create a Python environment

```
python3 -m venv .venv
source .venv/bin/activate
pip install --upgrade pip
pip install -r requirements.txt
```

If you use a web framework such as Flask or FastAPI for Slack endpoints, install it as part of your service dependencies as well.

---

6. Create the environment file

Create a `.env` file with your Slack and PinionAI credentials:

```
cp .env.example .env 2>/dev/null || touch .env
chmod 600 .env
nano .env
```

Example contents:

```
host_url=https://api.pinionai.com
client_id=<YOUR_CLIENT_ID>
client_secret=<YOUR_CLIENT_SECRET>
agent_id=<YOUR_AGENT_ID>

SLACK_BOT_TOKEN=xoxb-...
SLACK_SIGNING_SECRET=your-signing-secret
SLACK_CLIENT_ID=your-client-id
SLACK_CLIENT_SECRET=your-client-secret
```

Required variables

- `SLACK_BOT_TOKEN`: bot token for posting messages
- `SLACK_SIGNING_SECRET`: required for verifying Slack requests
- `SLACK_CLIENT_ID` / `SLACK_CLIENT_SECRET`: required for OAuth and installation flow
- `client_id` / `client_secret` / `agent_id`: your PinionAI agent configuration

> Keep the `.env` file private. Do not commit secrets to Git.

---

7. Configure the Slack app for Marketplace submission

In the Slack API dashboard, configure the app as follows:

A. Enable public distribution

1. Open Settings → App Distribution.
2. Turn on public distribution.
3. Fill in the app profile details, support URL, privacy policy URL, and app icon.

B. Configure Request URLs

Slack requires public HTTPS endpoints for:

- Event Subscriptions
- Interactivity & Shortcuts
- Slash Commands

Set the request URLs to the public domain you will expose through Nginx, for example:

- `https://your-domain.example.com/slack/events`
- `https://your-domain.example.com/slack/interactive`
- `https://your-domain.example.com/slack/commands`

C. Enable Event Subscriptions

1. Go to Features → Event Subscriptions.
2. Enable events.
3. Subscribe to the events your bot needs, such as:
    - `message.channels`
    - `message.groups`
    - `message.im`
4. Save and verify the Request URL.

D. Configure bot scopes

In Features → OAuth & Permissions, add the bot scopes your app needs, such as:

- `chat:write`
- `files:read`
- `channels:history`
- `groups:history`
- `im:history`

Install the app to your test workspace and verify the OAuth flow.

---

8. Create a public HTTPS endpoint

Because Slack Marketplace apps must use HTTP request URLs, your deployment should expose a web endpoint that receives Slack requests.

A typical layout is:

- `/slack/events` for events
- `/slack/interactive` for interactive components
- `/slack/commands` for slash commands

You can implement this with a lightweight Python web service such as Flask or FastAPI and run it behind a reverse proxy.

Example deployment pattern

- Run the web service locally on port `8000`
- Expose it through Nginx on port `80` and `443`
- Use a domain such as `slack.yourdomain.com`

---

9. Configure Nginx with HTTPS

If you already own a domain, point it to the VM and configure HTTPS.

Install and configure Nginx

```
sudo systemctl enable nginx
sudo systemctl start nginx
```

Create a site configuration:

```
sudo nano /etc/nginx/sites-available/slack-marketplace
```

Example configuration:

```
server {
    listen 80;
    server_name slack.yourdomain.com;

    location / {
        proxy_pass http://127.0.0.1:8000;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "upgrade";
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

Enable it:

```
sudo ln -s /etc/nginx/sites-available/slack-marketplace /etc/nginx/sites-
enabled/
sudo nginx -t
sudo systemctl reload nginx
```

Obtain HTTPS with Certbot

```
sudo certbot --nginx -d slack.yourdomain.com
```

This gives you a public HTTPS endpoint that Slack can verify.

---

10. Run the web service

A simple production pattern is to run the service with `gunicorn` or `uvicorn`.

Example with `uvicorn`:

```
source .venv/bin/activate
uvicorn app:app --host 127.0.0.1 --port 8000
```

If you prefer `gunicorn`:

```
source .venv/bin/activate
gunicorn -w 2 -k uvicorn.workers.UvicornWorker app:app --bind
127.0.0.1:8000
```

You should ensure the app is reachable at:

```
curl https://slack.yourdomain.com/slack/health
```

---

11. Create a systemd service

To keep the service running after reboots, create a systemd unit.

```
sudo nano /etc/systemd/system/pinionai-slack-marketplace.service
```

Example content:

```
[Unit]
Description=PinionAI Slack Marketplace App
After=network.target

[Service]
WorkingDirectory=/home/$USER/pinionai-streamlit-agent
EnvironmentFile=/home/$USER/pinionai-streamlit-agent/.env
ExecStart=/home/$USER/pinionai-streamlit-agent/.venv/bin/gunicorn -w 2 -k
uvicorn.workers.UvicornWorker app:app --bind 127.0.0.1:8000
Restart=always
RestartSec=5
User=$USER
Group=$USER

[Install]
WantedBy=multi-user.target
```

Enable and start it:

```
sudo systemctl daemon-reload
sudo systemctl enable pinionai-slack-marketplace
sudo systemctl start pinionai-slack-marketplace
sudo systemctl status pinionai-slack-marketplace
```

---

12. Validate the deployment

Before submitting the app to the Slack Marketplace, confirm that:

- the public HTTPS endpoint is reachable
- Slack can verify the Request URL
- the app is marked for public distribution
- the app profile metadata is complete
- your app passes a real workspace test

If Slack still reports submission errors, re-check:

- public distribution is enabled
- Socket Mode is not being used as the submission path
- your Request URLs are verified and reachable
- your app has a privacy policy and support URL

---

13. Summary

For a Slack Marketplace submission, the deployment must be public and HTTP-based. The VM setup is similar to the existing GCP e2-micro guide, but the important difference is that your app must expose a public HTTPS endpoint for Slack events and interactivity.